

# Armazenamento Remoto e Distribuído

Infraestruturas de Computação na Nuvem, Aula 11

---

**Catarina Silva**

ESTGA, Universidade de Aveiro | 2026

## Agenda (1/2)

---

- Armazenamento em rede: NAS vs. SAN.
- NFS e iSCSI em detalhe.
- Sistemas de ficheiros distribuídos.
- Ceph: arquitetura e componentes.
- Armazenamento de objetos e a API S3.

## Agenda (2/2)

---

- Consistência em sistemas distribuídos e o teorema CAP.
- Para além do CAP: o modelo PACELC.
- Replicação e sharding.
- Erasure coding vs replicação.
- Armazenamento distribuído no Proxmox e no Kubernetes.

## Armazenamento em Rede: NAS vs. SAN (1/2)

---

- **NAS** (Network Attached Storage): expõe armazenamento ao nível de **ficheiro**, através da rede.
- Protocolos típicos: **NFS** (ambientes Unix/Linux) e **SMB/CIFS** (ambientes Windows).
- Os clientes veem uma hierarquia de pastas e ficheiros, tal como num disco local partilhado.

## Armazenamento em Rede: NAS vs. SAN (2/2)

---

- **SAN** (Storage Area Network): expõe armazenamento ao nível de **bloco**, através de uma rede dedicada.
- Protocolos típicos: **iSCSI** (sobre TCP/IP) e **Fibre Channel** (rede dedicada de alto desempenho).
- O cliente vê o armazenamento como se fosse um disco local, formatando-o com o seu próprio sistema de ficheiros.
- NAS é mais simples de gerir; SAN tende a oferecer melhor desempenho e controlo mais fino.

## NFS em Detalhe

---

- O servidor *exporta* diretórios (definidos em `/etc/exports`); os clientes montam-nos como se fossem locais.
- Opções de exportação relevantes: `rw/ro` (leitura-escrita ou apenas leitura), `sync` (confirma escritas em disco antes de responder) e `root_squash` (mapeia o root remoto para um utilizador sem privilégios).
- **NFSv3** é sem estado; **NFSv4** introduz estado, melhor desempenho através de firewalls (porta única) e integração com Kerberos.
- As permissões baseiam-se em `UIDs/GIDs` numéricos: se não forem consistentes entre máquinas, os acessos comportam-se de forma inesperada.
- Vários clientes podem montar a mesma partilha simultaneamente – é o modelo natural de partilha de ficheiros.

## iSCSI em Detalhe

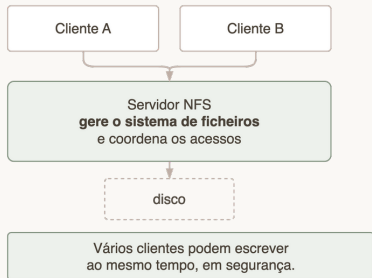
---

- Transporta comandos SCSI sobre TCP/IP, permitindo usar a rede existente em vez de uma rede Fibre Channel dedicada.
- **Target**: o servidor que disponibiliza o armazenamento. **Initiator**: o cliente que se liga a ele.
- **IQN** (iSCSI Qualified Name): identificador único de cada target e initiator, usado nas listas de controlo de acesso.
- **LUN** (Logical Unit Number): identifica cada unidade de armazenamento disponibilizada por um target.
- Diferença essencial face ao NFS: o cliente recebe um *dispositivo de bloco* em bruto, que tem de particionar e formatar.

# Quem Gere o Sistema de Ficheiros

---

## NAS — nível de FICHEIRO (NFS)



## SAN — nível de BLOCO (iSCSI)



# Sistemas de Ficheiros Distribuídos

---

- Motivação: um único servidor NAS/SAN tem limites de capacidade e desempenho.
- Sistemas de ficheiros distribuídos espalham os dados por várias máquinas, escalando além de um único nó.
- **HDFS** (Hadoop Distributed File System): pensado para grandes volumes de dados e processamento em lote.
- **Ceph** e **GlusterFS**: sistemas de armazenamento distribuído de uso geral, com replicação e tolerância a falhas.

## Ceph: Arquitetura

---

- Plataforma de armazenamento distribuído que oferece, sobre a mesma infraestrutura, acesso por **bloco**, **ficheiro** e **objeto**.
- **OSD** (Object Storage Daemon): processo responsável por um disco; guarda os dados e trata da replicação e recuperação.
- **MON** (Monitor): mantém o mapa do cluster e garante o consenso sobre o seu estado.
- **MDS** (Metadata Server): gere os metadados quando se usa o sistema de ficheiros CephFS.
- **CRUSH**: algoritmo que calcula onde cada objeto reside, dispensando uma tabela central de localização – é o que permite escalar sem um ponto de estrangulamento.

# Armazenamento de Objetos

---

- Modelo baseado em **buckets** (contentores) e **objetos** identificados por uma chave, com metadados associados.
- Acesso feito através de uma API HTTP, em vez de um protocolo de sistema de ficheiros ou de bloco.
- A API **S3** (Amazon Simple Storage Service) tornou-se a referência de facto do setor.
- **MinIO** é uma alternativa open-source compatível com a API S3, útil para on-premises ou testes locais.
- Casos de uso típicos: backups, dados não estruturados (imagens, logs) e conteúdo estático (websites).

# Consistência em Sistemas Distribuídos

---

- **Consistência forte:** todas as leituras devolvem sempre o valor mais recente escrito, em qualquer réplica.
- **Consistência eventual:** após uma escrita, as réplicas convergem para o mesmo valor, mas não de imediato.
- Teorema **CAP**: um sistema distribuído não pode garantir simultaneamente Consistency, Availability e Partition tolerance.
- Na prática, partições de rede acontecem, pelo que a escolha real é entre privilegiar consistência ou disponibilidade.

## Para Além do CAP: o Modelo PACELC

---

- O teorema CAP só descreve o comportamento *durante* uma partição de rede – uma situação relativamente rara.
- O modelo **PACELC** completa o quadro: se houver **P**artição, escolher entre **A**vailability e **C**onsistency; **E**m caso contrário (*else*), escolher entre **L**atência e **C**onsistência.
- Ou seja: mesmo em funcionamento normal, garantir consistência forte entre réplicas implica esperar por confirmações – e isso custa latência.
- Muitos sistemas de larga escala optam por consistência eventual precisamente por esta razão, não apenas por causa de partições.
- É um compromisso de desenho, não uma limitação técnica a ultrapassar.

# CAP e PACELC

---

Consistência  
todas as leituras veem  
o valor mais recente

Disponibilidade  
responde sempre,  
mesmo que desatualizado

Tolerância a partições  
a rede pode falhar  
— não é opcional

Havendo partição, escolhe-se entre C e A.

PACELC: se há Partição → A ou C; caso contrário (Else) → Latência ou Consistência

Mesmo sem partições há um custo: garantir consistência entre réplicas exige esperar por confirmações.

# Replicação e Sharding

---

- **Replicação**: mantém cópias dos mesmos dados em vários nós, aumentando a disponibilidade e a tolerância a falhas.
- **Sharding** (particionamento): divide os dados em partes distintas, distribuídas por vários nós, para aumentar a escala.
- As duas técnicas são complementares: sharding aumenta a capacidade total, replicação protege cada partição.
- Ambas introduzem desafios de consistência, geridos consoante o modelo escolhido (forte ou eventual).

## Erasure Coding vs Replicação

---

- **Replicação:** guardar  $n$  cópias completas. Simples e rápido a recuperar, mas dispendioso – 3 réplicas consomem 3 vezes o espaço.
- **Erasure coding:** dividir o dado em fragmentos e calcular fragmentos de paridade adicionais, distribuídos por vários nós.
- É a generalização do princípio do RAID 5/6 (Aula 10) para um sistema distribuído por muitas máquinas.
- Vantagem: tolerância a falhas equivalente com muito menos espaço ocupado.
- Custo: mais processamento e maior latência na recuperação, por ser necessário recalcular os dados em falta.
- Uso típico: replicação para dados quentes e frequentemente acedidos; erasure

# Armazenamento Distribuído no Proxmox e no Kubernetes

---

- O Proxmox integra suporte nativo para **NFS**, **iSCSI** e **Ceph** como tipos de storage.
- Armazenamento partilhado entre nós é o que torna possível a *migração ao vivo* de VMs (Aula 03): o disco não precisa de ser copiado.
- No Kubernetes, os **PersistentVolumes** (Aula 05) assentam nestes mesmos sistemas, através de drivers **CSI** (Container Storage Interface).
- **Modos de acesso**: `ReadWriteOnce` (um nó), `ReadOnlyMany` e `ReadWriteMany` (vários nós em escrita).
- Só armazenamento ao nível de ficheiro (ex. NFS, CephFS) suporta `ReadWriteMany` – um volume de bloco não pode ser escrito por vários nós, pela razão vista nos slides sobre iSCSI.

## Síntese da Aula (1/2)

---

- NAS (ficheiro, NFS/SMB) e SAN (bloco, iSCSI/Fibre Channel) são as duas formas clássicas de armazenamento em rede.
- Sistemas de ficheiros distribuídos (HDFS, Ceph, GlusterFS) escalam o armazenamento além de uma única máquina.
- Object storage (S3, MinIO) organiza dados em buckets e objetos, acedidos via API HTTP.

## Síntese da Aula (2/2)

---

- O teorema CAP obriga a escolher entre consistência e disponibilidade em caso de partição; o PACELC lembra que, mesmo sem partição, há um compromisso entre latência e consistência.
- Replicação garante disponibilidade; sharding garante escala; erasure coding oferece tolerância a falhas com menos espaço.
- O Ceph distribui dados sem controlador central (algoritmo CRUSH), evitando um single point of failure.
- Estes sistemas sustentam o storage partilhado do Proxmox (e a migração ao vivo) e os PersistentVolumes do Kubernetes via CSI.

# Obrigada.

---

*Questões e comentários*